

Setup

From a clean machine to a running agent. Sections 1 and 2 are all you need to see it work.

1. Get the credentials

Five accounts. All have a free tier, and the first three are enough for the offline demo and a browser call. Everything goes into `.env`.

LiveKit	<code>cloud.livekit.io</code>	<i>Settings, then Keys</i> The WebSocket URL, the API key and the secret. The secret is shown once, when the key is created.
Opik	<code>comet.com/signup</code>	<i>Account settings, then API keys</i> The API key, plus your workspace name, which is your Comet username unless you made a team.
OpenAI	<code>platform.openai.com</code>	<i>API keys</i> Drives the conversation and the post-call analysis.
Deepgram	<code>console.deepgram.com</code>	<i>API keys</i> Transcription. Optional: without it, speech falls back to LiveKit Inference.
ElevenLabs	<code>elevenlabs.io</code>	<i>Profile menu, then API keys</i> The voice. Optional. A free plan can only use premade voices through the API.

Only three are required

LiveKit, Opik and one model provider. Without Deepgram or ElevenLabs the agent still runs: speech falls back to LiveKit Inference, which is billed through your LiveKit account and needs no extra key.

2. Install and configure

```
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate     # macOS / Linux

pip install -r requirements.txt
python agent.py download-files  # one-off: the turn-detector model

cp .env.example .env           # then fill in your keys
```

Windows and OneDrive

Do not create the virtualenv inside a OneDrive-synced folder. OneDrive locks files during install and pip fails with WinError 32. Put it somewhere like `C:\Users\you\.venvs\agent`.

Values still left as template text, anything containing `xxxx` or `your-`, are detected and treated as unset, so a half-filled file gives a clear message naming the variable rather than a confusing 401 from a provider.

Running it

Three ways, in increasing order of what they need.

A. Offline demo — no phone, no browser, no worker

Replays three scripted calls through the real analysis and Opik code. The fastest way to see the whole post-call flow and get traces into Opik.

```
python demo_pipeline.py --all
python demo_pipeline.py --scenario hallucination
```

The hallucination scenario is a deliberately bad call: the agent invents a normal HbA1c and suggests stopping a medication. Biomarker fidelity and medical safety should both collapse.

B. Talk to it in a browser — needs LiveKit

```
# terminal 1, leave running
python agent.py dev

# terminal 2
python dispatch_call.py --patient PT-10432 --simulate
python join_link.py
```

join_link.py prints a URL. Open it, allow the microphone, and the agent speaks first. The room stays open 30 minutes with nobody in it.

C. Ring a real telephone — needs a SIP trunk

LiveKit does not sell phone numbers, so you bring a carrier. With a Twilio account that owns a voice-capable number, one command does the whole setup.

```
python setup_telephony.py --check
python setup_telephony.py --number +1XXXXXXXXXX --apply
# paste the printed ST... id into .env as SIP_OUTBOUND_TRUNK_ID

python dispatch_call.py --patient PT-10432 --phone +91XXXXXXXXXX
```

It creates the trunk, generates its credentials, attaches the number and registers it with LiveKit. Without --apply it prints exactly what it would create and changes nothing.

Twilio trial accounts

You can only call numbers verified in the console, and you may have exactly one SIP trunk. If one already exists the script reuses it rather than deleting it. Calling India also needs geographic permissions enabled under Voice settings.

After a call

transcripts/<call-id>.json

Transcript, tool calls, analysis and telemetry

recordings/<call-id>.ogg

The call audio

Opik project healthcare-voice-agent

The trace, spans, scores and the attached recording

Other commands

python dispatch_call.py --list

The patients on file

python join_link.py --list

Active rooms

python -m pytest tests -q

47 tests, no network or API keys

python agent.py console

Local microphone, no LiveKit room

python opik_integration.py

Create the server-side Opik evaluation rules

Reference

What each file does, and what to do when something misbehaves.

The files

<code>agent.py</code>	The LiveKit worker: prompt, seven function tools, SIP dialling, and the post-call pipeline.
<code>opik_integration.py</code>	Standalone observability and evaluation. Imports nothing from this project.
<code>post_call_analysis.py</code>	Outcome and booking. Deterministic where it can be, a model where it must be.
<code>dispatch_call.py</code>	Starts a call by creating a room and dispatching the agent to it.
<code>booking_service.py</code>	Simulated clinic scheduler. One slot is always taken, so rejection is exercised.
<code>call_data.py</code>	Patient, biomarker and call-context models, plus environment loading.
<code>demo_pipeline.py</code>	Offline end-to-end demo and artifact replay.
<code>setup_telephony.py</code>	One-shot Twilio to LiveKit SIP trunk setup, with a dry-run mode.
<code>join_link.py</code>	Prints a browser join link for a simulated call.
<code>tests/test_pipeline.py</code>	47 tests. The Opik client is replaced with a recording double.
<code>data/patients.json</code>	Two sample patients with realistic biomarker panels.

When something misbehaves

Failed to dispatch the agent

The worker is not running, or `AGENT_NAME` differs between worker and dispatcher.

A call never connects, and the worker log is silent

Stale worker processes are stealing dispatches. Kill every python running `agent.py` and start one.

The agent is in no room when you join

The room expired. Dispatch again and join within the window.

402 paid_plan_required from ElevenLabs

A free plan cannot use shared library voices through the API. Pick one whose category is premade.

model_not_found from a provider

Available models change. List them from the provider's API and set the matching env var.

rate_limit_exceeded during evaluation

A free tier's token rate. The judges retry with the delay the provider states; it resolves itself.

Evaluation scores missing

The LLM judges need a model key. Without one the two deterministic evaluators still run.

WinError 32 during pip install

OneDrive is locking the virtualenv. Create it outside any synced folder.

Turn taking is left at the SDK defaults

The endpointing environment variables exist but should stay unset. Overriding them, together with preemptive generation, stopped turns completing: the greeting played, the patient's reply was transcribed, and the agent never answered because it was waiting for a turn the SDK had already delivered.